

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240029 123240199
NAMA	:	TAZKYA SYAKIEB DWININGTYAS ANINDYA YOLA PUSPITA
PLUG	:	IF- B
NAMA ASISTEN	:	BINTORO NAURA RIFDA NUR RAMADHANI

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI**

**UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Tazkya Syakieb Dwiningtyas 123240029

Anindya Yola Puspita 123240199

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

BINTORO

**NAURAH RIFDAH NUR
RAMADHANI**

NIM. 123230059

NIM. 123230068

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Pendukung Keputusan Penentuan Prioritas Peserta Program Diet Menggunakan Metode Weighted Product (WP). Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 11 Mei 2026

Penyusun 1,

Penyusun 2,

Tazkya Syakieb Dwiningtyas
NIM. 1232240029

Anindya Yola Puspita
NIM. 123240199

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	2
1.3 Manfaat Proyek Akhir	2
BAB II PEMBAHASAN.....	4
2.1 Dasar Teori	4
2.2 Skenario Kasus & Dataset	5
2.3 Pemodelan Sistem Pendukung Keputusan	Error! Bookmark not defined.
2.4 Perhitungan & Algoritma	6
2.5 Implementasi & Tampilan Sistem	8
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS.....	31
3.1 Jadwal Pengerjaan	31
3.2 Pembagian Tugas.....	31
BAB IV KESIMPULAN DAN SARAN.....	32
4.2 Kesimpulan.....	32
4.3 Saran	32
DAFTAR PUSTAKA.....	32

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi informasi saat ini telah memberikan banyak kemudahan dalam berbagai bidang kehidupan, termasuk dalam bidang kesehatan. Pemanfaatan teknologi tidak hanya digunakan sebagai sarana penyimpanan data, tetapi juga sebagai alat bantu dalam proses pengambilan keputusan yang membutuhkan ketelitian dan pertimbangan dari berbagai faktor. Salah satu penerapan teknologi tersebut adalah Sistem Pendukung Keputusan (SPK), yaitu sistem yang dirancang untuk membantu pengguna dalam menentukan keputusan terbaik berdasarkan sejumlah kriteria yang telah ditetapkan. Program diet merupakan salah satu upaya yang banyak dilakukan untuk meningkatkan kualitas kesehatan masyarakat. Diet tidak hanya bertujuan untuk menurunkan berat badan, tetapi juga untuk mengendalikan berbagai kondisi kesehatan seperti obesitas, kadar kolesterol tinggi, tekanan darah yang tidak normal, serta kadar glukosa yang berlebihan. Dalam pelaksanaannya, sering kali terdapat banyak calon peserta yang memiliki kondisi kesehatan yang berbeda-beda sehingga diperlukan proses seleksi untuk menentukan peserta yang lebih diprioritaskan dalam mengikuti program diet. Penentuan prioritas peserta program diet secara manual memiliki beberapa kelemahan, seperti membutuhkan waktu yang cukup lama, rentan terhadap kesalahan perhitungan, serta memungkinkan terjadinya subjektivitas dalam pengambilan keputusan. Selain itu, banyaknya kriteria yang harus dipertimbangkan membuat proses evaluasi menjadi semakin kompleks. Oleh karena itu, diperlukan suatu sistem yang mampu membantu proses penilaian secara cepat, akurat, dan objektif. Pada proyek akhir ini digunakan metode Weighted Product (WP) sebagai metode pengambilan keputusan. Metode Weighted Product dipilih karena mampu melakukan proses perankingan alternatif berdasarkan banyak kriteria dengan mempertimbangkan tingkat kepentingan masing-masing kriteria melalui pembobotan. Metode ini juga memiliki proses perhitungan yang relatif sederhana namun mampu menghasilkan keputusan yang efektif dan objektif. Kriteria yang digunakan dalam penelitian ini meliputi Body Mass Index (BMI), kadar kolesterol, tekanan darah, kadar glukosa, jam olahraga per minggu, tingkat ketaatan diet,

dan skor ketidakseimbangan gizi. Seluruh kriteria tersebut digunakan sebagai dasar dalam menentukan tingkat prioritas peserta program diet. Data kemudian diolah menggunakan metode Weighted Product sehingga menghasilkan nilai preferensi yang digunakan untuk menyusun ranking peserta. Berdasarkan uraian tersebut, maka dibuat sebuah Sistem Pendukung Keputusan Penentuan Prioritas Peserta Program Diet Menggunakan Metode Weighted Product (WP) berbasis web dengan memanfaatkan framework Streamlit. Sistem ini diharapkan dapat membantu proses seleksi peserta program diet secara lebih cepat, objektif, dan terukur sehingga keputusan yang dihasilkan menjadi lebih efektif dan sesuai dengan kondisi kesehatan masing-masing peserta.

1.2 Tujuan Proyek Akhir

Tujuan yang ingin dicapai dalam proyek akhir ini adalah sebagai berikut:

1. Merancang dan membangun Sistem Pendukung Keputusan untuk menentukan prioritas peserta program diet berdasarkan beberapa kriteria kesehatan.
2. Mengimplementasikan metode Weighted Product (WP) ke dalam sistem sebagai metode perhitungan untuk menghasilkan ranking peserta program diet.
3. Membantu proses pengambilan keputusan agar dapat dilakukan secara lebih cepat, objektif, dan akurat dibandingkan dengan proses penilaian secara manual.
4. Menyajikan hasil perhitungan dan perankingan peserta dalam bentuk aplikasi berbasis web yang mudah digunakan oleh pengguna.
5. Menampilkan visualisasi data kesehatan peserta sehingga pengguna dapat memperoleh informasi yang lebih mudah dipahami dalam proses pengambilan keputusan.

1.3 Manfaat Proyek Akhir

Manfaat yang diharapkan dari proyek akhir ini adalah sebagai berikut:

1. Bagi Pengguna

Sistem yang dibangun dapat membantu pengguna dalam menentukan prioritas peserta program diet secara lebih cepat dan objektif. Pengguna tidak perlu lagi melakukan perhitungan secara manual karena seluruh proses penilaian dilakukan secara otomatis oleh sistem berdasarkan data yang tersedia.

2. Bagi Instansi atau Pengelola Program Diet

Sistem dapat digunakan sebagai alat bantu dalam melakukan proses seleksi peserta program diet secara lebih efektif dan efisien. Hasil perankingan yang dihasilkan sistem dapat dijadikan sebagai dasar dalam menentukan peserta yang memiliki prioritas lebih tinggi untuk mendapatkan program diet terlebih dahulu.

3. Bagi Pengembang

Proyek akhir ini dapat menjadi sarana untuk menerapkan ilmu yang telah dipelajari selama perkuliahan, khususnya pada bidang Sistem Pendukung Keputusan, pengolahan data, dan pengembangan aplikasi berbasis web menggunakan Python dan Streamlit.

4. Bagi Akademik

Proyek akhir ini dapat menjadi referensi bagi mahasiswa lain yang ingin mempelajari penerapan metode Weighted Product (WP) dalam Sistem Pendukung Keputusan serta implementasinya dalam bentuk aplikasi berbasis web.

5. Bagi Pengembangan Ilmu Pengetahuan

Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan teknologi informasi khususnya pada penerapan Sistem Pendukung Keputusan di bidang kesehatan, sehingga dapat digunakan sebagai dasar untuk penelitian selanjutnya yang berkaitan dengan proses pengambilan keputusan multikriteria.

BAB II PEMBAHASAN

2.1 Dasar Teori

2.1.1 Sistem Pendukung Keputusan (SPK)

Sistem Pendukung Keputusan (SPK) atau Decision Support System (DSS) merupakan suatu sistem berbasis komputer yang dirancang untuk membantu pengguna dalam proses pengambilan keputusan. Sistem ini memanfaatkan data, model, dan metode tertentu untuk menghasilkan informasi yang dapat digunakan sebagai bahan pertimbangan dalam menentukan keputusan yang paling tepat. SPK tidak menggantikan peran pengambil keputusan, melainkan berfungsi sebagai alat bantu yang mampu memberikan rekomendasi berdasarkan hasil pengolahan data secara sistematis dan objektif.

Dalam bidang kesehatan, Sistem Pendukung Keputusan dapat digunakan untuk membantu proses analisis data pasien, menentukan tingkat prioritas penanganan, melakukan evaluasi kondisi kesehatan, serta membantu proses seleksi peserta pada suatu program kesehatan. Penggunaan SPK dapat mengurangi tingkat subjektivitas dalam pengambilan keputusan karena seluruh proses dilakukan berdasarkan data dan kriteria yang telah ditetapkan sebelumnya.

Pada proyek akhir ini, Sistem Pendukung Keputusan digunakan untuk menentukan prioritas peserta program diet berdasarkan kondisi kesehatan masing-masing pasien. Sistem akan mengolah data kesehatan yang terdapat pada dataset dan menghasilkan peringkat peserta yang dapat dijadikan sebagai dasar dalam menentukan siapa yang lebih diprioritaskan untuk mengikuti program diet.

2.1.2 Metode Weighted Product (WP)

Weighted Product (WP) merupakan salah satu metode dalam Sistem Pendukung Keputusan yang termasuk ke dalam kelompok Multi Attribute Decision Making (MADM). Metode ini digunakan untuk menentukan alternatif terbaik berdasarkan sejumlah kriteria yang telah diberikan bobot sesuai tingkat kepentingannya.

Prinsip utama metode Weighted Product adalah melakukan perkalian terhadap seluruh nilai atribut yang dimiliki oleh setiap alternatif. Setiap nilai atribut dipangkatkan terlebih dahulu

menggunakan bobot kriteria yang telah dinormalisasi. Dengan cara tersebut, metode WP mampu memperhitungkan tingkat kepentingan masing-masing kriteria dalam proses pengambilan keputusan.

Metode Weighted Product memiliki dua jenis kriteria yaitu kriteria benefit dan kriteria cost. Kriteria benefit merupakan kriteria yang nilainya semakin besar maka semakin baik, sedangkan kriteria cost merupakan kriteria yang nilainya semakin kecil maka semakin baik. Pada sistem yang dibangun, kriteria jam olahraga per minggu dan ketaatan diet termasuk ke dalam kategori benefit, sedangkan BMI, kolesterol, tekanan darah, glukosa, dan skor ketidakseimbangan gizi termasuk ke dalam kategori cost.

Tahapan perhitungan metode Weighted Product meliputi proses normalisasi bobot, perhitungan vektor S, perhitungan vektor V, dan proses perankingan. Hasil akhir dari metode ini berupa nilai preferensi yang digunakan untuk menentukan urutan prioritas setiap pasien dalam program diet.

2.2 Skenario Kasus & Dataset

2.2.1 Skenario Kasus

Program diet merupakan salah satu upaya yang dilakukan untuk membantu individu mencapai kondisi kesehatan yang lebih baik. Namun dalam praktiknya, jumlah peserta yang ingin mengikuti program diet sering kali cukup banyak sehingga diperlukan proses seleksi untuk menentukan peserta yang memiliki tingkat prioritas lebih tinggi.

Penentuan prioritas peserta tidak dapat dilakukan hanya berdasarkan satu indikator kesehatan saja. Seorang pasien mungkin memiliki BMI yang tinggi, tetapi memiliki kadar glukosa yang normal. Sebaliknya, pasien lain mungkin memiliki kadar glukosa dan kolesterol yang tinggi sehingga membutuhkan perhatian yang lebih besar. Oleh karena itu, diperlukan suatu sistem yang mampu mempertimbangkan berbagai faktor kesehatan secara bersamaan. Pada proyek akhir ini, dibuat sebuah Sistem Pendukung Keputusan untuk menentukan prioritas peserta program diet menggunakan metode Weighted Product. Sistem melakukan evaluasi terhadap setiap pasien berdasarkan tujuh kriteria kesehatan yang meliputi BMI, kadar kolesterol, tekanan darah, kadar glukosa, jam olahraga per minggu, tingkat ketaatan diet, dan skor ketidakseimbangan gizi. Seluruh kriteria tersebut digunakan sebagai dasar dalam menentukan tingkat prioritas pasien.

Hasil akhir dari sistem berupa ranking pasien berdasarkan nilai preferensi yang diperoleh dari proses perhitungan metode Weighted Product. Pasien yang memiliki nilai preferensi tertinggi akan memperoleh prioritas utama untuk mengikuti program diet karena dianggap memiliki tingkat kebutuhan yang lebih besar dibandingkan pasien lainnya.

2.2.2 Dataset

Dataset yang digunakan dalam sistem berasal dari file `diet_prioritas.csv` yang berisi data sebanyak 250 pasien. Setiap data pasien memiliki sejumlah atribut kesehatan yang digunakan sebagai kriteria dalam proses pengambilan keputusan.

Adapun kriteria yang digunakan dalam sistem adalah sebagai berikut:

Kode	Kriteria	Jenis
C1	BMI	Cost
C2	Kolesterol	Cost
C3	Tekanan Darah	Cost
C4	Glukosa	Cost
C5	Jam Olahraga per Minggu	Benefit
C6	Ketaatan Diet	Benefit
C7	Skor Ketidakseimbangan Gizi	Cost

Data tersebut diproses menggunakan metode Weighted Product sehingga menghasilkan nilai preferensi untuk setiap pasien. Nilai preferensi tersebut kemudian digunakan untuk menentukan urutan prioritas peserta program diet.

2.3 Perhitungan & Algoritma

2.3.1 Perhitungan

Pada sistem dilakukan menggunakan metode Weighted Product sesuai dengan implementasi yang terdapat pada program. Proses perhitungan dimulai dengan membaca dataset pasien menggunakan library Pandas. Setelah data berhasil dimuat, sistem menentukan bobot dan jenis kriteria yang digunakan dalam proses pengambilan keputusan.

Langkah pertama adalah melakukan normalisasi bobot. Proses ini dilakukan agar total seluruh bobot bernilai satu sehingga dapat digunakan dalam perhitungan metode Weighted Product. Bobot yang telah dinormalisasi kemudian digunakan sebagai pangkat pada setiap nilai kriteria.

Langkah berikutnya adalah menghitung nilai Vektor S. Pada tahap ini, setiap nilai kriteria pasien dipangkatkan menggunakan bobot yang telah dinormalisasi. Untuk kriteria benefit digunakan bobot positif, sedangkan untuk kriteria cost digunakan bobot negatif. Seluruh hasil perpangkatan kemudian dikalikan sehingga menghasilkan nilai Vektor S untuk setiap pasien.

Setelah nilai Vektor S diperoleh, sistem menghitung nilai Vektor V. Nilai V diperoleh dengan membagi setiap nilai Vektor S dengan jumlah seluruh nilai Vektor S yang ada pada dataset. Nilai V merupakan nilai preferensi yang digunakan untuk menentukan tingkat prioritas setiap pasien.

Tahap terakhir adalah proses perankingan. Sistem mengurutkan seluruh pasien berdasarkan nilai V dari yang terbesar hingga yang terkecil. Pasien yang memiliki nilai V tertinggi akan menempati posisi pertama dan menjadi peserta dengan prioritas tertinggi dalam program diet.

2.3.2 Algoritma

Secara umum algoritma sistem dapat dijelaskan sebagai berikut:

1. Membaca dataset pasien dari file CSV.
2. Menentukan bobot dan jenis kriteria.
3. Melakukan normalisasi bobot.
4. Menghitung nilai Vektor S untuk setiap pasien.
5. Menghitung nilai Vektor V.
6. Mengurutkan nilai V secara menurun.
7. Menentukan ranking pasien.
8. Menampilkan hasil perhitungan dan visualisasi data pada aplikasi Streamlit.

Dengan menggunakan algoritma tersebut, proses penentuan prioritas peserta program diet dapat dilakukan secara lebih cepat, objektif, dan konsisten dibandingkan dengan proses penilaian secara manual.

2.4 Implementasi & Tampilan Sistem

File Main.py

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
import io

# -----
# KONFIGURASI HALAMAN
# -----
st.set_page_config(
    page_title="SPK Prioritas Program Diet",
    page_icon="🍌",
    layout="wide",
    initial_sidebar_state="collapsed"
)

# -----
# GLOBAL CSS
# -----
st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700;800&family=Space+Mono:wght@400;700&display=swap');

/* — Palet Warna
#FCFCFC near-white (base background)
#F1DBE2 blush pink (card background)
#EBB0C0 dusty rose (aksen, border)
#7DAC95 sage green (highlight, aksen)
#684F51 muted mauve/brown (teks utama)
— */

/* — Reset & Base — */
html, body, [class*="css"] {
    font-family: 'Plus Jakarta Sans', sans-serif;
    color: #684F51;
}
.stApp {
    background:#transparent;
}

/* — Hide default streamlit chrome — */
#MainMenu, footer, header { visibility: hidden; }
.block-container {
    padding-top: 0 !important;
    padding-bottom: 2rem;
    max-width: 1400px;
}
```

```

/* — Hero Banner — */
.hero-wrap {
  background:
    radial-gradient(ellipse at top right, rgba(125,172,149,0.35) 0%,
transparent 50%),
    radial-gradient(ellipse at bottom left, rgba(235,176,192,0.4)
0%, transparent 50%),
    linear-gradient(135deg, #FCFCFC 0%, #EBB0C0 55%, #7DAC95 100%);
  border-bottom: 1px solid #d4a8b8;
  padding: 3rem 3rem 2.5rem;
  border-radius: 0 0 20px 20px;
  margin: 0rem -1rem 2rem;
  position: relative;
  overflow: hidden;
  text-align: center;
}
.hero-wrap::before { content: none; }
.hero-wrap::after { content: none; }

.hero-badge {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  background: rgba(125,172,149,0.2);
  border: 1px solid rgba(125,172,149,0.55);
  color: #4a8a6f;
  font-size: 0.72rem;
  font-weight: 600;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  padding: 5px 14px;
  border-radius: 100px;
  margin-bottom: 1rem;
}

.hero-title {
  font-size: 2.4rem;
  font-weight: 800;
  color: #684F51;
  line-height: 1.2;
  margin: 0 0 0.5rem;
}

.hero-title span {
  background: linear-gradient(90deg, #7DAC95, #4a8a6f);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

.hero-sub {
  font-size: 0.95rem;
  color: #684F51;
  opacity: 0.65;
  font-weight: 400;
  margin: 0;
}

/* — Stat Cards — */
.stat-row {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

```

```

    margin-bottom: 1.5rem;
}
.stat-card {
    background: #7DAC95;
    border: 1px solid #EBB0C0;
    border-radius: 14px;
    padding: 1.25rem 1.5rem;
    display: flex;
    align-items: center;
    gap: 1rem;
    transition: border-color 0.2s, box-shadow 0.2s;
}
.stat-card:hover { border-color: #7DAC95; box-shadow: 0 4px 16px
rgba(125,172,149,0.15); }
.stat-icon {
    width: 46px; height: 46px;
    background: rgba(235,176,192,0.2);
    border-radius: 12px;
    display: flex; align-items: center; justify-content: center;
    font-size: 1.4rem;
    flex-shrink: 0;
}
.stat-label {
    font-size: 0.75rem;
    color: #9c7a80;
    font-weight: 500;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    margin-bottom: 3px;
}
.stat-value {
    font-size: 1.6rem;
    font-weight: 700;
    color: #684F51;
    font-family: 'Space Mono', monospace;
    line-height: 1;
}

/* — Section Header — */
.section-header {
    display: flex;
    align-items: center;
    gap: 10px;
    margin: 1.5rem 0 1rem;
}
.section-header .sh-icon {
    width: 36px; height: 36px;
    background: rgba(125,172,149,0.18);
    border-radius: 9px;
    display: flex; align-items: center; justify-content: center;
    font-size: 1.1rem;
}
.section-header .sh-text {
    font-size: 1.05rem;
    font-weight: 700;
    color: #684F51;
}
.section-header .sh-sub {
    font-size: 0.78rem;

```

```

        color: #9c7a80;
        margin-top: 1px;
    }

/* — Tabs — */
.stTabs [data-baseweb="tab-list"] {
    background: #7DAC95;
    border-radius: 12px;
    padding: 5px;
    gap: 4px;
    border: 1px solid #EBB0C0;
    justify-content: center;
    width: fit-content;
    margin: 0 auto;
}
.stTabs [data-baseweb="tab"] {
    background: transparent;
    border-radius: 9px;
    color: #9c7a80;
    font-weight: 600;
    font-size: 0.85rem;
    padding: 8px 18px;
    transition: all 0.2s;
}
.stTabs [aria-selected="true"] {
    background: rgba(125,172,149,0.25) !important;
    color: #4a8a6f !important;
}
.stTabs [data-baseweb="tab-panel"] {
    padding-top: 1.5rem;
}

/* — Dataframe — */
.stDataFrame {
    border: 1px solid #EBB0C0 !important;
    border-radius: 12px !important;
    overflow: hidden;
}

/* — Expander — */
.streamlit-expanderHeader {
    background: #7DAC95 !important;
    border: 1px solid #EBB0C0 !important;
    border-radius: 10px !important;
    color: #684F51 !important;
    font-weight: 600 !important;
}
.streamlit-expanderContent {
    background: #7DAC95 !important;
    border: 1px solid #EBB0C0 !important;
    border-top: none !important;
    border-radius: 0 0 10px 10px !important;
}

/* — Buttons — */
.stButton > button {
    background: linear-gradient(135deg, #7DAC95, #5a9a7a);
    color: white;
    border: none;

```

```

border-radius: 10px;
font-weight: 700;
font-size: 0.9rem;
padding: 0.65rem 1.5rem;
transition: all 0.2s;
box-shadow: 0 4px 15px rgba(125,172,149,0.3);
}
.stButton > button:hover {
background: linear-gradient(135deg, #5a9a7a, #4a8a6f);
transform: translateY(-1px);
box-shadow: 0 6px 20px rgba(125,172,149,0.4);
}
.stButton > button[kind="primary"] {
background: linear-gradient(135deg, #c0606e, #d4788a) !important;
box-shadow: 0 4px 15px rgba(192,96,110,0.25) !important;
}

/* — Download Button — */
.stDownloadButton > button {
background: #7DAC95;
border: 1px solid #EBB0C0;
color: #684F51;
border-radius: 10px;
font-weight: 600;
transition: all 0.2s;
}
.stDownloadButton > button:hover {
border-color: #EBB0C0;
background: rgba(125,172,149,0.08);
color: #4a8a6f;
}

/* — Inputs & Selects — */
.stTextInput input, .stNumberInput input, .stSelectbox select {
background: #7DAC95 !important;
border: 1px solid #EBB0C0 !important;
border-radius: 9px !important;
color: #684F51 !important;
}
.stTextInput input:focus, .stNumberInput input:focus {
border-color: #7DAC95 !important;
box-shadow: 0 0 0 2px rgba(125,172,149,0.2) !important;
}
.stMultiSelect [data-baseweb="select"] {
background: #7DAC95 !important;
border-color: #EBB0C0 !important;
}

/* — Slider — */
.stSlider [data-baseweb="slider"] {
padding-top: 0.5rem;
}
.stSlider [data-baseweb="thumb"] {
background: #7DAC95 !important;
border-color: #7DAC95 !important;
}
.stSlider [data-baseweb="track"] > div:first-child {
background: #F1DBE2 !important;
}

```



```

.stSlider [data-baseweb="track"] > div:nth-child(2) {
  background: #7DAC95 !important;
}

/* — Metric — */
[data-testid="metric-container"] {
  background: #7DAC95;
  border: 1px solid #EBB0C0;
  border-radius: 12px;
  padding: 1rem 1.25rem;
}
[data-testid="metric-container"] label {
  color: #9c7a80 !important;
  font-size: 0.78rem !important;
  font-weight: 600 !important;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}
[data-testid="metric-container"] [data-testid="stMetricValue"] {
  color: #684F51 !important;
  font-family: 'Space Mono', monospace !important;
  font-size: 1.4rem !important;
}
[data-testid="metric-container"] [data-testid="stMetricDelta"] {
  font-size: 0.8rem !important;
}

/* — Divider — */
hr {
  border-color: #EBB0C0 !important;
  margin: 1.5rem 0 !important;
}

/* — Info / Warning / Success — */
.stAlert {
  border-radius: 10px !important;
}

/* — Popup — */
@keyframes slideInDown {
  from { transform: translateY(-18px); opacity: 0; }
  to   { transform: translateY(0);      opacity: 1; }
}
@keyframes fadeOut {
  0%   { opacity: 1; } 70% { opacity: 1; } 100% { opacity: 0; }
}
.crud-popup {
  display: flex; align-items: center; gap: 0.75rem;
  margin-top: 0.75rem; padding: 0.85rem 1.2rem;
  border-radius: 10px; font-size: 0.92rem; font-weight: 500;
  box-shadow: 0 4px 20px rgba(104,79,81,0.15);
  animation: slideInDown 0.35s cubic-bezier(.22,1,.36,1), fadeOut 3.5s
ease forwards;
  width: 100%; pointer-events: none;
}
.crud-popup.success {
  background: linear-gradient(135deg,#d4ede3,#b8ddc8);
  border-left: 4px solid #7DAC95;
  color: #2d6a4f;
}

```

```

.crud-popup.error {
  background: linear-gradient(135deg, #f5d0d8, #eea8b8);
  border-left: 4px solid #EBB0C0;
  color: #7a2535; }
.crud-popup.warning {
  background: linear-gradient(135deg, #fdf0d0, #f8dfa0);
  border-left: 4px solid #e6c060;
  color: #7a5c10; }
.crud-popup .popup-icon { font-size: 1.5rem; flex-shrink: 0; }
.crud-popup .popup-title { font-size: 1rem; font-weight: 700; margin-bottom: 2px; }
.crud-popup .popup-body { font-size: 0.85rem; opacity: 0.9; }

/* — Rank Cards — */
.rank-card {
  background: #7DAC95;
  border: 1px solid #EBB0C0;
  border-radius: 14px;
  padding: 1.25rem;
  text-align: center;
  transition: all 0.2s;
}
.rank-card:hover { border-color: #7DAC95; transform: translateY(-2px);
box-shadow: 0 6px 20px rgba(125,172,149,0.15); }
.rank-medal { font-size: 2rem; margin-bottom: 0.4rem; }
.rank-label { font-size: 0.7rem; color: #9c7a80; font-weight: 600; text-transform: uppercase; letter-spacing: 0.05em; }
.rank-id { font-size: 1.3rem; font-weight: 800; color: #684F51; font-family: 'Space Mono', monospace; margin: 4px 0; }
.rank-score { font-size: 0.78rem; color: #4a8a6f; font-family: 'Space Mono', monospace; }

/* — Bobot Card — */
.bobot-card {
  background: #7DAC95;
  border: 1px solid #EBB0C0;
  border-radius: 12px;
  padding: 1rem 0.75rem;
  text-align: center;
  margin-bottom: 0.5rem;
}
.bobot-kode { font-size: 0.7rem; color: #9c7a80; font-weight: 700; text-transform: uppercase; letter-spacing: 0.06em; }
.bobot-tipe { display: inline-block; font-size: 0.65rem; padding: 2px 8px; border-radius: 100px; margin: 4px 0; font-weight: 600; }
.bobot-tipe.cost { background: rgba(235,176,192,0.25); color: #a8485a; border: 1px solid rgba(235,176,192,0.6); }
.bobot-tipe.benefit { background: rgba(125,172,149,0.2); color: #4a8a6f; border: 1px solid rgba(125,172,149,0.45); }
.bobot-nama { font-size: 0.72rem; color: #9c7a80; line-height: 1.3; margin-top: 4px; }

/* — Priority Banner — */
.priority-banner {
  background: linear-gradient(135deg, #f1dbe2 0%, #f5e8ed 100%);
  border: 1px solid #EBB0C0;
  border-left: 4px solid #7DAC95;
  border-radius: 14px;
  padding: 1.5rem 2rem;
  margin-bottom: 1.5rem;
  display: flex;

```

```

        align-items: center;
        gap: 1.5rem;
    }
    .priority-trophy { font-size: 2.5rem; }
    .priority-label { font-size: 0.72rem; color: #9c7a80; font-weight: 600;
text-transform: uppercase; letter-spacing: 0.06em; margin-bottom: 4px; }
    .priority-id { font-size: 1.6rem; font-weight: 800; color: #684F51;
font-family: 'Space Mono', monospace; }
    .priority-detail { font-size: 0.82rem; color: #4a8a6f; margin-top: 4px;
}

/* — Member Card — */
.member-card {
    background: #7DAC95;
    border: 1px solid #EBB0C0;
    border-radius: 14px;
    padding: 1.5rem;
    text-align: center;
    transition: all 0.25s;
}
.member-card:hover { border-color: #7DAC95; box-shadow: 0 0 24px
rgba(125,172,149,0.15); }
.member-avatar {
    width: 60px; height: 60px;
    background: linear-gradient(135deg, #EBB0C0, #7DAC95);
    border-radius: 50%;
    display: flex; align-items: center; justify-content: center;
    font-size: 1.6rem;
    margin: 0 auto 1rem;
}
.member-name { font-size: 1rem; font-weight: 700; color: #684F51; margin-
bottom: 4px; }
.member-nim { font-size: 0.78rem; color: #4a8a6f; font-family: 'Space
Mono', monospace; }

/* — Info Table — */
.info-table {
    background: #7DAC95;
    border: 1px solid #EBB0C0;
    border-radius: 14px;
    overflow: hidden;
}
.info-row {
    display: flex;
    border-bottom: 1px solid #f2d0da;
    padding: 0.9rem 1.5rem;
}
.info-row:last-child { border-bottom: none; }
.info-key { color: #9c7a80; font-size: 0.82rem; font-weight: 600;
width: 160px; flex-shrink: 0; }
.info-val { color: #684F51; font-size: 0.82rem; }

/* — Norm badge — */
.norm-row {
    display: flex;
    gap: 0.5rem;
    flex-wrap: wrap;
    margin-top: 0.75rem;
    margin-bottom: 1rem;
}

```

```

}
.norm-badge {
    background: #7DAC95;
    border: 1px solid #EBB0C0;
    border-radius: 8px;
    padding: 6px 14px;
    font-size: 0.78rem;
    color: #9c7a80;
}
.norm-badge span { color: #4a8a6f; font-family: 'Space Mono', monospace;
font-weight: 700; }
</style>
"""', unsafe_allow_html=True)

# -----
# HELPER: POPUP
# -----
def show_popup(title: str, body: str, kind: str = "success"):
    icons = {"success": "✅", "error": "❌", "warning": "⚠️"}
    icon = icons.get(kind, "❓")
    st.markdown(
        f'<div class="crud-popup {kind}">'
        f'<span class="popup-icon">{icon}</span>'
        f'<div class="popup-text">'
        f'<div class="popup-title">{title}</div>'
        f'<div class="popup-body">{body}</div>'
        f'</div></div>',
        unsafe_allow_html=True
    )

# -----
# KONSTANTA KRITERIA
# -----
KRITERIA = {
    "C1": {"nama": "BMI", "tipe": "cost", "kolom": "BMI"},
    "C2": {"nama": "Kolesterol (mg/dL)", "tipe": "cost", "kolom": "Cholesterol_mg/dL"},
    "C3": {"nama": "Tekanan Darah (mmHg)", "tipe": "cost", "kolom": "Blood_Pressure_mmHg"},
    "C4": {"nama": "Glukosa (mg/dL)", "tipe": "cost", "kolom": "Glucose_mg/dL"},
    "C5": {"nama": "Jam Olahraga/Minggu", "tipe": "benefit", "kolom": "Weekly_Exercise_Hours"},
    "C6": {"nama": "Ketaatan Rencana Diet (%)", "tipe": "benefit", "kolom": "Adherence_to_Diet_Plan"},
    "C7": {"nama": "Skor Ketidakseimbangan Gizi", "tipe": "cost", "kolom": "Dietary_Nutrient_Imbalance_Score"},
}

KOLOM_CRUD = [
    "Patient_ID", "BMI", "Cholesterol_mg/dL", "Blood_Pressure_mmHg",
    "Glucose_mg/dL", "Weekly_Exercise_Hours", "Adherence_to_Diet_Plan",
    "Dietary_Nutrient_Imbalance_Score", "Diet_Recommendation",
]

LABEL_CRUD = {
    "Patient_ID": "ID Pasien",

```

```

    "BMI": "C1 - BMI",
    "Cholesterol_mg/dL": "C2 - Kolesterol (mg/dL)",
    "Blood_Pressure_mmHg": "C3 - Tekanan Darah (mmHg)",
    "Glucose_mg/dL": "C4 - Glukosa (mg/dL)",
    "Weekly_Exercise_Hours": "C5 - Jam Olahraga/Minggu",
    "Adherence_to_Diet_Plan": "C6 - Ketaatan Diet (%)",
    "Dietary_Nutrient_Imbalance_Score": "C7 - Skor Ketidakseimbangan
Gizi",
    "Diet_Recommendation": "Rekomendasi Diet",
}

BOBOT_DEFAULT =
{"C1":25,"C2":15,"C3":15,"C4":20,"C5":10,"C6":5,"C7":10}

# Warna palette plotly (konsisten) - blush/sage light theme
COLORS =
["#EBB0C0","#7DAC95","#F1DBE2","#9c7a80","#4a8a6f","#d4788a","#b8ddc8"]

# -----
# LOAD DATA
# -----
@st.cache_data
def load_data(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    for info in KRITERIA.values():
        df[info["kolom"]] = df[info["kolom"]].clip(lower=0.001)
    return df.reset_index(drop=True)

CRUD_PATH = "diet_kriteria_edited.csv"

def save_crud_data(df): df[KOLOM_CRUD].to_csv(CRUD_PATH, index=False)

def load_crud_data(df_raw):
    try:
        df_e = pd.read_csv(CRUD_PATH)
        for info in KRITERIA.values():
            col = info["kolom"]
            if col in df_e.columns:
                df_e[col] = df_e[col].clip(lower=0.001)
        return df_e
    except FileNotFoundError:
        return df_raw.copy()

def init_session_data(df_raw):
    if "df_crud" not in st.session_state:
        st.session_state["df_crud"] = load_crud_data(df_raw)

def df_to_csv_bytes(df):
    buf = io.StringIO(); df.to_csv(buf, index=False)
    return buf.getvalue().encode()

# -----
# ALGORITMA WP
# -----
def hitung_wp(df, bobot_raw):
    total_bobot = sum(bobot_raw.values())
    bobot = {k: v/total_bobot for k,v in bobot_raw.items()}
    hasil = df[["Patient_ID"]].copy()
    S = np.ones(len(df))

```

```

        for kode, info in KRITERIA.items():
            nilai = df[info["kolom"]].values.astype(float)
            w = bobot[kode]
            S *= nilai**w if info["tipe"]=="benefit" else nilai**(-w)
        hasil["Vektor S"] = np.round(S, 6)
        V = S / S.sum()
        hasil["Vektor V"] = np.round(V, 6)
        hasil = hasil.sort_values("Vektor V",
ascending=False).reset_index(drop=True)
        hasil.index += 1
        hasil.insert(0, "Ranking", hasil.index)
        for kode, info in KRITERIA.items():
            hasil[info["nama"]] =
df.set_index("Patient_ID").loc[hasil["Patient_ID"],
info["kolom"]].values
            if "Diet_Recommendation" in df.columns:
                hasil["Rekomendasi Diet"] =
df.set_index("Patient_ID").loc[hasil["Patient_ID"],
"Diet_Recommendation"].values
            return hasil

# -----
# PLOTLY THEME (dark, konsisten)
# -----
PLOT_LAYOUT = dict(
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(241,219,226,0.3)",
    font=dict(family="Plus Jakarta Sans", color="#684F51", size=12),
    margin=dict(l=20, r=20, t=55, b=30),
)

# -----
# CRUD
# -----
def render_crud(df_work):
    df = df_work.copy()

    if "_popup" in st.session_state:
        p = st.session_state.pop("_popup")
        show_popup(p[0], p[1], p[2])

    # Tabel
    st.markdown("""
<div class="section-header">
    <div class="sh-icon">📄</div>
    <div><div class="sh-text">Data Peserta Program Diet</div>
    <div class="sh-sub">Semua kolom kriteria ditampilkan</div></div>
</div>""", unsafe_allow_html=True)

    tampil =
df[KOLOM_CRUD].rename(columns=LABEL_CRUD).reset_index(drop=True)
    tampil.index += 1

    filter_id = st.multiselect("Filter ID Pasien",
options=sorted(df["Patient_ID"].tolist()),
key="crud_filter")
    if filter_id:

```

```

tampil =
df[df["Patient_ID"].isin(filter_id)][KOLOM_CRUD].rename(columns=LABEL_C
RUD).reset_index(drop=True)
tampil.index += 1

st.dataframe(tampil, use_container_width=True, height=340)
st.divider()

# Tambah
with st.expander("****Tambah*** Data Pasien Baru"):
    c1, c2 = st.columns(2)
    new_id = c1.text_input("ID Pasien (contoh: P0251)",
key="add_id")
    new_c1 = c1.number_input("C1 - BMI",
value=22.0, step=0.1, key="add_c1")
    new_c2 = c2.number_input("C2 - Kolesterol
(mg/dL)",
min_value=0.0, value=180.0, step=1.0, key="add_c2")
    new_c3 = c1.number_input("C3 - Tekanan Darah
(mmHg)",
min_value=0.0, value=120.0, step=1.0, key="add_c3")
    new_c4 = c2.number_input("C4 - Glukosa
(mg/dL)",
min_value=0.0, value=100.0,
step=1.0, key="add_c4")
    new_c5 = c1.number_input("C5 - Jam Olahraga per
Minggu",
min_value=0.0, value=3.0, step=0.5, key="add_c5")
    new_c6 = c2.number_input("C6 - Ketaatan Rencana Diet
(%)",
min_value=0.0, max_value=100.0, value=70.0, step=1.0,
key="add_c6")
    new_c7 = c1.number_input("C7 - Skor Ketidakseimbangan Gizi",
min_value=0.0, value=2.0, step=0.1, key="add_c7")
    new_rek = c2.text_input("Rekomendasi Diet", value="Balanced",
key="add_rek")
    if st.button("Simpan Data Baru", key="btn_add"):
        if not new_id.strip():
            show_popup("Gagal", "ID Pasien tidak boleh kosong.",
"error")
        elif new_id.strip() in df["Patient_ID"].values:
            show_popup("Gagal", f"ID '{new_id.strip()}' sudah ada.",
"error")
        else:
            row =
{"Patient_ID":new_id.strip(),"BMI":max(new_c1,0.001),"Cholesterol_mg/dL
":max(new_c2,0.001),
"Blood_Pressure_mmHg":max(new_c3,0.001),"Glucose
_mg/dL":max(new_c4,0.001),
"Weekly_Exercise_Hours":max(new_c5,0.001),"Adher
ence_to_Diet_Plan":max(new_c6,0.001),
"Dietary_Nutrient_Imbalance_Score":max(new_c7,0.
001),"Diet_Recommendation":new_rek.strip()}
            df = pd.concat([df, pd.DataFrame([row])],
ignore_index=True)
            st.session_state["df_crud"] = df; save_crud_data(df)
            st.session_state["_popup"] = ("Berhasil Ditambahkan!",
f"Pasien '{new_id.strip()}' tersimpan.", "success")
            st.rerun()

st.divider()

# Edit

```

```

with st.expander("***Edit*** Data Pasien"):
    id_edit = st.selectbox("Pilih ID Pasien yang akan diedit:",
sorted(df["Patient_ID"].tolist()), key="edit_id_select")
    if id_edit:
        row = df[df["Patient_ID"]==id_edit].iloc[0]
        c1, c2 = st.columns(2)
        e1 = c1.number_input("C1 - BMI",
value=float(row["BMI"]),
step=0.1, key="edit_c1")
        e2 = c2.number_input("C2 - Kolesterol (mg/dL)",
value=float(row["Cholesterol_mg/dL"]),
step=1.0, key="edit_c2")
        e3 = c1.number_input("C3 - Tekanan Darah (mmHg)",
value=float(row["Blood_Pressure_mmHg"]),
step=1.0, key="edit_c3")
        e4 = c2.number_input("C4 - Glukosa (mg/dL)",
value=float(row["Glucose_mg/dL"]),
step=1.0, key="edit_c4")
        e5 = c1.number_input("C5 - Jam Olahraga per Minggu",
value=float(row["Weekly_Exercise_Hours"]),
step=0.5, key="edit_c5")
        e6 = c2.number_input("C6 - Ketaatan Rencana Diet (%)",
value=float(row["Adherence_to_Diet_Plan"]),
step=1.0, key="edit_c6")
        e7 = c1.number_input("C7 - Skor Ketidakseimbangan Gizi",
value=float(row["Dietary_Nutrient_Imbalance_Score"]),
step=0.1, key="edit_c7")
        er = c2.text_input("Rekomendasi Diet",
value=str(row["Diet_Recommendation"]), key="edit_rek")
        if st.button("Simpan Perubahan", key="btn_edit"):
            idx = df[df["Patient_ID"]==id_edit].index[0]
            df.at[idx, "BMI"]=max(e1,0.001);
df.at[idx, "Cholesterol_mg/dL"]=max(e2,0.001)
df.at[idx, "Blood_Pressure_mmHg"]=max(e3,0.001);
df.at[idx, "Glucose_mg/dL"]=max(e4,0.001)
df.at[idx, "Weekly_Exercise_Hours"]=max(e5,0.001);
df.at[idx, "Adherence_to_Diet_Plan"]=max(e6,0.001)
df.at[idx, "Dietary_Nutrient_Imbalance_Score"]=max(e7,0.001);
df.at[idx, "Diet_Recommendation"]=er.strip()
            st.session_state["df_crud"]=df; save_crud_data(df)
            st.session_state["popup"] = ("Data Diperbarui!", f"Pasien '{id_edit}' berhasil diubah.", "success")
            st.rerun()

st.divider()

# Hapus
with st.expander("***Hapus*** Data Pasien"):
    id_hapus = st.selectbox("Pilih ID Pasien yang akan dihapus:",
sorted(df["Patient_ID"].tolist()), key="hapus_id_select")
    st.warning(f"Data pasien **{id_hapus}** akan dihapus dari sesi ini.")
    if st.button("Hapus Data Ini", key="btn_hapus", type="primary"):
        df = df[df["Patient_ID"]!=id_hapus].reset_index(drop=True)
        st.session_state["df_crud"]=df; save_crud_data(df)
        st.session_state["popup"] = ("Data Dihapus!", f"Pasien '{id_hapus}' telah dihapus.", "warning")
        st.rerun()

```



```

    st.divider()
    st.download_button("***Klik untuk Unduh Dataset Pasien Diet***",
                        data=df_to_csv_bytes(df[KOLOM_CRUD].rename(columns=LABEL_CRUD)),
                        file_name="dataset_diet_kriteria.csv",
                        mime="text/csv", key="dl_crud")
    return df

# -----
# MAIN
# -----
def main():
    # -----
    # Hero
    # Banner

    st.markdown("""
    <div class="hero-wrap">
        <div class="hero-badge">🍎 Sistem Pendukung Keputusan</div>
        <div class="hero-title">Prioritas Peserta <span>Program
Diet</span></div>
        <!-- <p class="hero-sub">Metode Weighted Product (WP)
&nbsp;·&nbsp;&nbsp; 7 Kriteria Kesehatan &nbsp;·&nbsp;&nbsp; 250 Data Pasien</p> --
    >
    </div>
    """, unsafe_allow_html=True)

    DATA_PATH = "diet_prioritas.csv"
    try:
        df_raw = load_data(DATA_PATH)
        init_session_data(df_raw)
    except FileNotFoundError:
        st.error(f"File `{DATA_PATH}` tidak ditemukan. Pastikan berada
di folder yang sama.")
        st.stop()

    tab1, tab2, tab3, tab4 = st.tabs([
        " **Dataset Pasien** ",
        " **Perhitungan SPK** ",
        " **Analisis Visual** ",
        " **Profil Kelompok** ",
    ])

    # -----
    # TAB
    # 1

    with tab1:
        df_sesi = st.session_state.get("df_crud", df_raw)
        if not isinstance(df_sesi, pd.DataFrame):
            df_sesi = df_raw.copy(); st.session_state["df_crud"] = df_sesi

        # Stat cards
        st.markdown(f"""
        <div class="stat-row">
            <div class="stat-card">
                <div class="stat-icon">👤</div>
                <div><div class="stat-label">Total Pasien</div>
                <div class="stat-value">{len(df_sesi)}</div></div>
            </div>
            <div class="stat-card">
                <div class="stat-icon">📊</div>
                <div><div class="stat-label">Jumlah Kriteria</div>

```

```

        <div class="stat-value">{len(KRITERIA)}</div></div>
    </div>
    <div class="stat-card">
        <div class="stat-icon">📊</div>
        <div><div class="stat-label">Metode</div>
        <div class="stat-value">Weighted Product</div></div>
    </div>
</div>
""" , unsafe_allow_html=True)

with st.expander("Statistik Ringkas Kriteria"):
    kolom_k = [info["kolom"] for info in KRITERIA.values()]
    label_m = {info["kolom"]: info["nama"] for info in
KRITERIA.values()}
    st.dataframe(df_sesi[kolom_k].rename(columns=label_m).descr
ibe().round(2), use_container_width=True)

    st.divider()
    df_updated = render_crud(df_sesi)
    st.session_state["df_crud"] = df_updated

# — TAB 2

with tab2:
    st.markdown("""
    <div class="section-header">
        <div class="sh-icon">🏠</div>
        <div><div class="sh-text">Konfigurasi Bobot Kriteria</div>
        <div class="sh-sub">Total bobot akan dinormalisasi otomatis
- tidak harus 100</div></div>
    </div>""", unsafe_allow_html=True)

    df_untuk_wp = st.session_state.get("df_crud", df_raw)

    # Bobot cards + sliders
    cols = st.columns(7)
    bobot_input = {}
    for i, (kode, info) in enumerate(KRITERIA.items()):
        with cols[i]:
            tipe_cls = info["tipe"]
            tipe_label = "▲ Benefit" if tipe_cls=="benefit" else
" ▼ Cost"

            st.markdown(f"""
            <div class="bobot-card">
                <div class="bobot-kode">{kode}</div>
                <div class="bobot-tipe {tipe_cls}">{tipe_label}</div>
                <div class="bobot-nama">{info['nama']}</div>
            </div>""", unsafe_allow_html=True)
            bobot_input[kode] = st.slider("", 1, 100,
BOBOT_DEFAULT[kode], 1, key=f"sl_{kode}", label_visibility="collapsed")

    # Bobot ternormalisasi
    total_brt = sum(bobot_input.values())
    norm_html = "".join([
                                f'<div class="norm-badge">{k}:
<span>{v/total_brt:.3f}</span></div>'
                                for k, v in bobot_input.items()
        ])

```

```

        st.markdown(f'<div style="font-size:0.78rem;color:#4d7a65;font-
weight:600;text-transform:uppercase;letter-spacing:.05em;margin-
bottom:6px">Bobot          Ternormalisasi</div><div          class="norm-
row">{norm_html}</div>', unsafe_allow_html=True)

        st.divider()

        if st.button("**Hitung Prioritas (WP)**",
use_container_width=True):
            with st.spinner("Menghitung Vektor S dan Vektor V ..."):
                hasil_wp = hitung_wp(df_untuk_wp, bobot_input)
                st.session_state["hasil_wp"] = hasil_wp
                st.session_state["bobot_input"] = bobot_input

            if "hasil_wp" in st.session_state:
                hasil_wp = st.session_state["hasil_wp"]
                bobot_input = st.session_state["bobot_input"]
                top1 = hasil_wp.iloc[0]

                # Priority banner
                rek = top1.get("Rekomendasi Diet", "-")
                st.markdown(f"""
<div class="priority-banner">
    <div class="priority-trophy">🏆</div>
    <div class="priority-label">Prioritas Utama Program
Diet</div>
    <div class="priority-id">{top1['Patient_ID']}</div>
    <div class="priority-detail">Vektor V: {top1['Vektor
V']:.6f} &nbsp; &nbsp; &nbsp; Rekomendasi: {rek}</div>
</div>
""", unsafe_allow_html=True)

                # Top 3 rank cards
                st.markdown(f'<div style="font-
size:0.78rem;color:#9c7a80;font-weight:600;text-
transform:uppercase;letter-spacing:.05em;margin-bottom:10px">Top 3
Peserta Prioritas</div>', unsafe_allow_html=True)
                mc = st.columns(3)
                medals = [("🥇", "Peringkat 1"), ("🥈", "Peringkat
2"), ("🥉", "Peringkat 3")]
                for i, (col, (medal, label)) in enumerate(zip(mc, medals)):
                    row = hasil_wp.iloc[i]
                    col.markdown(f"""
<div class="rank-card">
    <div class="rank-medal">{medal}</div>
    <div class="rank-label">{label}</div>
    <div class="rank-id">{row['Patient_ID']}</div>
    <div class="rank-score">V = {row['Vektor
V']:.6f}</div>
</div>""", unsafe_allow_html=True)

                st.divider()

                st.markdown(f"""
<div class="section-header">
    <div class="sh-icon">📄</div>

```

```

<div><div class="sh-text">Tabel Hasil Perankingan Semua
Pasien</div></div>
</div>""", unsafe_allow_html=True)

tampil_cols = ["Ranking", "Patient_ID", "Vektor S", "Vektor V"]
+ [info["nama"] for info in KRITERIA.values()]
if "Rekomendasi Diet" in hasil_wp.columns:
    tampil_cols.append("Rekomendasi Diet")
st.dataframe(hasil_wp[tampil_cols].reset_index(drop=True),
use_container_width=True, height=480)

st.download_button("***Klik untuk Unduh Hasil Perankingan
Prioritas Pasien***", data=df_to_csv_bytes(hasil_wp),
                    file_name="hasil_perankingan_diet_wp.csv",
mime="text/csv", key="dl_hasil")
else:
    st.info("Atur bobot kriteria di atas lalu klik **Hitung
Prioritas (WP)** untuk melihat hasil.")

```

#	—	TAB	3
---	---	-----	---

```

with tab3:
    if "hasil_wp" not in st.session_state:
        st.warning("⚠ Jalankan perhitungan WP di tab **Perhitungan
SPK** terlebih dahulu.")
    else:
        hasil_wp = st.session_state["hasil_wp"]
        bobot_input = st.session_state["bobot_input"]

        # Grafik 1 - Bar top10
        st.markdown("""<div class="section-header">
            <div class="sh-icon"><img alt="Bar chart icon" data-bbox="525 525 545 540"/></div>
            <div><div class="sh-text">Top 10 Pasien Prioritas
Tertinggi</div></div>
            </div>""", unsafe_allow_html=True)
        top10 = hasil_wp.head(10).sort_values("Vektor V",
ascending=True)
        fig_bar = go.Figure(go.Bar(
            x=top10["Vektor V"], y=top10["Patient_ID"],
orientation="h",
            marker=dict(color=top10["Vektor V"],
colorscale=[[0, "#EBB0C0"], [0.5, "#7DAC95"], [1, "#4a8a6f"]],
showscale=True, colorbar=dict(title="Skor
V", tickfont=dict(color="#684F51"))),
            text=[f"{v:.5f}" for v in top10["Vektor V"]],
textposition="outside",
            textfont=dict(color="#684F51"),
            hovertemplate="<b>{%y}</b><br>Vektor V:
{%x:.6f}<extra></extra>"
        ))
        fig_bar.update_layout(**PLOT_LAYOUT,
title=dict(text="Perankingan Pasien berdasarkan Vektor V",
font=dict(color="#684F51", size=15)),
            xaxis=dict(title="Nilai Vektor V",
gridcolor="rgba(235,176,192,0.5)", color="#684F51"),
            yaxis=dict(title="ID Pasien",
color="#684F51"), height=420)
        st.plotly_chart(fig_bar, use_container_width=True)

```

```

st.divider()

# Grafik 2 - Pie bobot (dua kolom)
col_pie, col_bar2 = st.columns([1, 1])
with col_pie:
    st.markdown("""<div class="section-header">
        <div class="sh-icon">🍪</div>
        <div><div class="sh-text">Distribusi Bobot
Kriteria</div></div>
        </div>""", unsafe_allow_html=True)
    total_b = sum(bobot_input.values())
    bobot_pct = {k: round(v/total_b*100,1) for k,v in
bobot_input.items()}
    fig_pie = go.Figure(go.Pie(
        labels=[f"{k}: {KRITERIA[k]['nama']}" for k in
bobot_pct],
        values=list(bobot_pct.values()), hole=0.45,
        marker=dict(colors=COLORS,
line=dict(color="#FCFCFC",width=2)),
        textinfo="label+percent",
        hovertemplate="<b>{%label}</b><br>{%value}%<extra><
/extra>"
    ))
    fig_pie.update_layout(**PLOT_LAYOUT, height=380,
        legend=dict(orientation="v", x=1.02,
y=0.5, font=dict(color="#684F51",size=10)))
    st.plotly_chart(fig_pie, use_container_width=True)

with col_bar2:
    st.markdown("""<div class="section-header">
        <div class="sh-icon">📊</div>
        <div><div class="sh-text">Sebaran Vektor V Semua
Pasien</div></div>
        </div>""", unsafe_allow_html=True)
    fig_hist = go.Figure(go.Histogram(
        x=hasil_wp["Vektor V"], nbinsx=30,
        marker=dict(color="#7DAC95",
line=dict(color="#EBB0C0",width=1)),
        hovertemplate="Rentang:  {%x}<br>Jumlah:
{%y}<extra></extra>"
    ))
    fig_hist.update_layout(**PLOT_LAYOUT, height=380,
        xaxis=dict(title="Vektor V",
gridcolor="rgba(235,176,192,0.5)", color="#684F51"),
        yaxis=dict(title="Jumlah Pasien",
gridcolor="rgba(235,176,192,0.5)", color="#684F51"))
    st.plotly_chart(fig_hist, use_container_width=True)

st.divider()

# Grafik 3 - Scatter BMI vs V
st.markdown("""<div class="section-header">
    <div class="sh-icon">📊</div>
    <div><div class="sh-text">Korelasi BMI vs Skor Akhir
(Vektor V)</div></div>
    </div>""", unsafe_allow_html=True)
fig_scatter = px.scatter(
    hasil_wp, x="BMI", y="Vektor V", text="Patient ID",

```

```

color="Vektor V",
color_continuous_scale=[[0,"#EBB0C0"],[0.5,"#7DAC95"],[1,"#4a8a6f"]],
    hover_data={"Patient_ID":True,"Ranking":True,"Vektor
V":":.6f"}
    )
    fig_scatter.update_traces(textposition="top center",
textfont=dict(size=8,color="#684F51"),
                                marker=dict(size=8,
line=dict(color="#FCFCFC",width=1)))
    fig_scatter.update_layout(**PLOT_LAYOUT, height=460,
                                xaxis=dict(title="BMI",gridcolor=
"rgba(235,176,192,0.5)",color="#684F51"),
                                yaxis=dict(title="Vektor
V",gridcolor="rgba(235,176,192,0.5)",color="#684F51"),
                                coloraxis_colorbar=dict(title="Skor
V",tickfont=dict(color="#684F51")))
    st.plotly_chart(fig_scatter, use_container_width=True)
    st.divider()

    # Grafik 4 - Heatmap
    st.markdown("""<div class="section-header">
    <div class="sh-icon">🔥</div>
    <div><div class="sh-text">Heatmap Nilai Kriteria - Top
10 Pasien</div></div>
    </div>""", unsafe_allow_html=True)
    nama_k = [info["nama"] for info in KRITERIA.values()]
    heat_df = hasil_wp.head(10).set_index("Patient_ID")[nama_k]
    heat_norm = (heat_df - heat_df.min()) / (heat_df.max() -
heat_df.min() + 1e-9)
    fig_heat = px.imshow(heat_norm, text_auto=".2f",
color_continuous_scale="RdYlGn", aspect="auto")
    fig_heat.update_layout(**PLOT_LAYOUT, height=420,
                                coloraxis_colorbar=dict(title="Norm.
",tickfont=dict(color="#684F51")))
    fig_heat.update_traces(textfont=dict(size=10,color="#684F51
"))
    st.plotly_chart(fig_heat, use_container_width=True)

# — TAB 4

with tab4:
    st.markdown("""
    <div class="section-header" style="margin-top:0.5rem">
    <div class="sh-icon">👤</div>
    <div><div class="sh-text">Profil Kelompok</div>
    <div class="sh-sub">SPK Penentuan Prioritas Peserta Program
Diet</div></div>
    </div>""", unsafe_allow_html=True)

    # Info proyek
    st.markdown("""
    <div class="info-table" style="margin-bottom:1.5rem">
    <div class="info-row"><div class="info-key">Mata
Kuliah</div><div class="info-val">Praktikum Sistem Cerdas dan Pendukung
Keputusan</div></div>
    <div class="info-row"><div class="info-key">Metode</div><div
class="info-val">Weighted Product (WP)</div></div>

```

```

        <div class="info-row"><div class="info-key">Dataset</div><div
class="info-val">diet_prioritas.csv      -      250      data      pasien,      7
kriteria</div></div>
        <div class="info-row"><div class="info-key">Tools</div><div
class="info-val">Python · Streamlit · Pandas · NumPy · Plotly</div></div>
        <div class="info-row"><div class="info-key">Tahun
Akademik</div><div class="info-val">2025 / 2026</div></div>
    </div>
    """ , unsafe_allow_html=True)

    # Anggota
    st.markdown(' <div style="font-size:0.78rem;color:#9c7a80;font-
weight:600;text-transform:uppercase;letter-spacing:.05em;margin-
bottom:12px">Anggota Kelompok</div> ', unsafe_allow_html=True)

    anggota = [
        {"nama": "Tazkya Syakieb Dwiningtyas", "nim": "123240029"},
        {"nama": "Anindya Yola Puspita", "nim": "123240199"},
    ]

    mc = st.columns(len(anggota))
    for col, data in zip(mc, anggota):
        inisial = "".join([w[0] for w in
data["nama"].split()[1:2]]).upper()
        col.markdown(f"""
        <div class="member-card">
            <div class="member-avatar">{inisial}</div>
            <div class="member-name">{data['nama']}</div>
            <div class="member-nim">{data['nim']}</div>
        </div>""", unsafe_allow_html=True)

    st.divider()

    # Penjelasan kriteria
    st.markdown("""
    <div class="section-header">
        <div class="sh-icon">✍️</div>
        <div><div class="sh-text">Keterangan Kriteria WP</div></div>
    </div>""", unsafe_allow_html=True)

    rows_html = ""
    for kode, info in KRITERIA.items():
        tipe_cls = info["tipe"]
        tipe_label = "Benefit" if tipe_cls=="benefit" else "Cost"
        rows_html += f"""
        <div class="info-row">
            <div class="info-key">{kode} - {info['nama']}</div>
            <div class="info-val">
                <span class="bobot-tipe {tipe_cls}" style="margin-
right:8px">{tipe_label}</span>
                Bobot default: <span style="color:#4a8a6f;font-
family:'Space Mono',monospace">{BOBOT_DEFAULT[kode]}</span>
            </div>
        </div>""""
        st.markdown(f' <div class="info-table">{rows_html}</div> ',
unsafe_allow_html=True)

if __name__ == "__main__":
    main()

```

(tangkapan layar sistem)

1. Menu Profil Kelompok

SISTEM PENDUKUNG KEPUTUSAN

Prioritas Peserta Program Diet

Dataset PasienPerhitungan SPKAnalisis VisualProfil Kelompok

Profil Kelompok

SPK Penentuan Prioritas Peserta Program Diet

Mata Kuliah

Praktikum Sistem Cerdas dan Pendukung Keputusan

Metode

Weighted Product (WP)

Dataset

diet_prioritas.csv — 250 data pasien, 7 kriteria

Tools

Python - Streamlit - Pandas - NumPy - Plotly

Tahun Akademik

2025 / 2026

2. Menu Profil Kelompok Anggota Kelompok

ANGGOTA KELOMPOK

TS

Tazkya Syakieb Dwiningtyas

123240029

AY

Anindya Yola Puspita

123240199

3. Menu DataSet

SISTEM PENDUKUNG KEPUTUSAN

Prioritas Peserta Program Diet

Dataset PasienPerhitungan SPKAnalisis VisualProfil Kelompok

TOTAL PASIEN

250

JUMLAH KRITERIA

7

METODE

Weighted Product

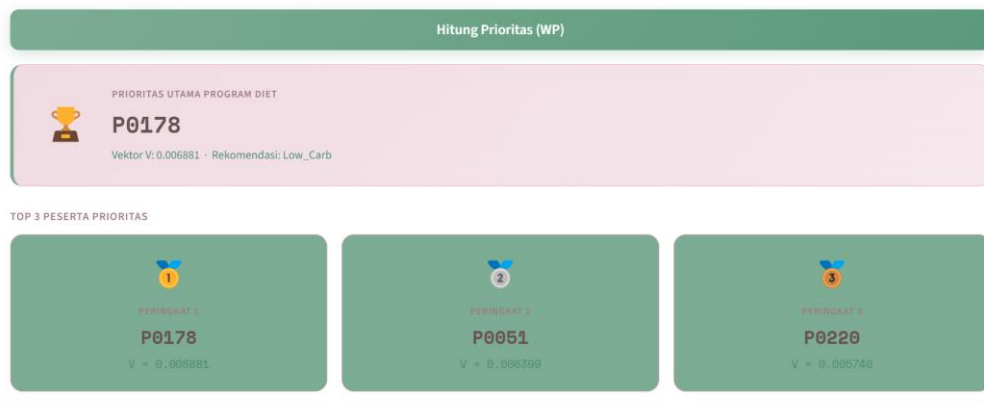
Statistik Ringkas Kriteria

	BMI	Kolesterol (mg/dL)	Tekanan Darah (mmHg)	Glukosa (mg/dL)	Jam Olahraga/Minggu	Ketaatan Rencana Diet (%)	Skor Ketidakseimbangan Gizi
count	250	250	250	250	250	250	250
mean	28.4	199.66	144.41	141.03	5.1	74.34	2.52
std	8.1	29.37	20.51	38.04	2.86	14.38	1.47
min	13.3	151.7	110	70.7	0	50.2	0

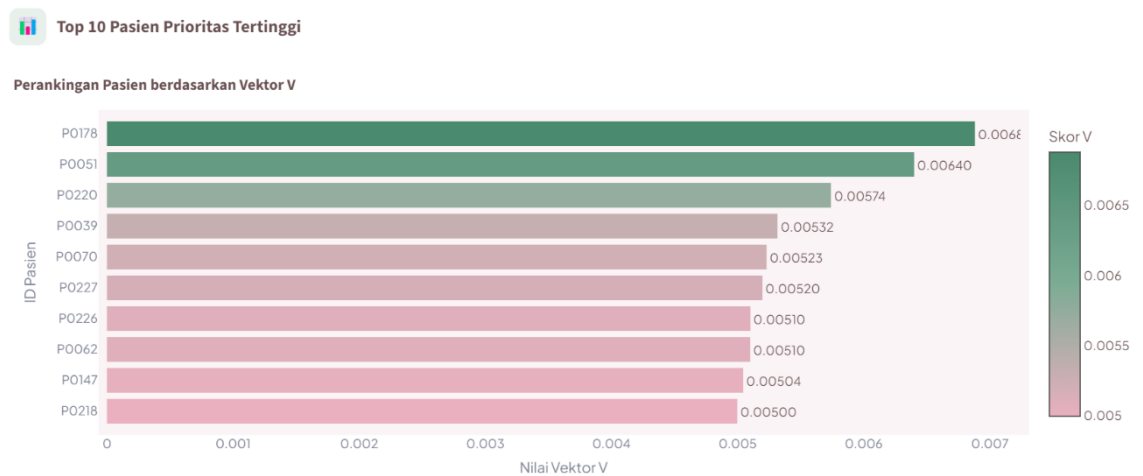
4. Menu Hitung SPK



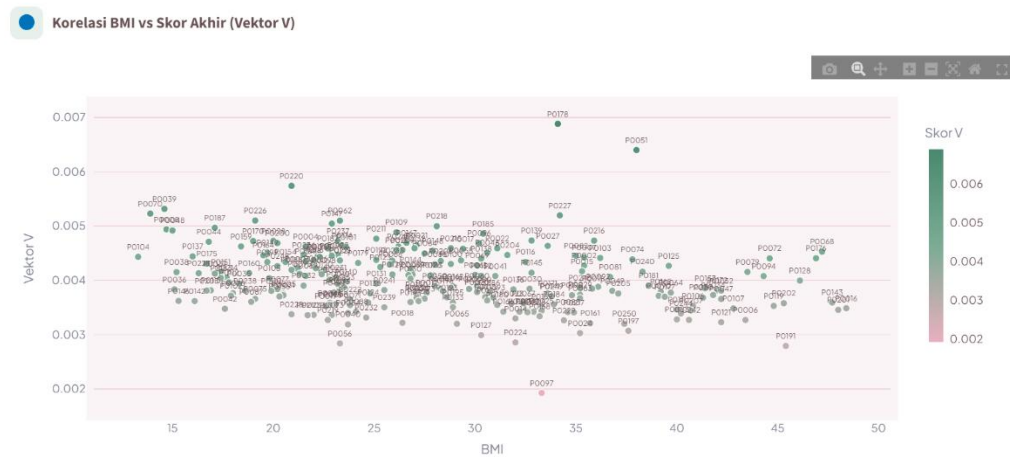
5. Menu Hitung SPK (Rincian Peringkat dan Detail Langkah Perhitungan WP)



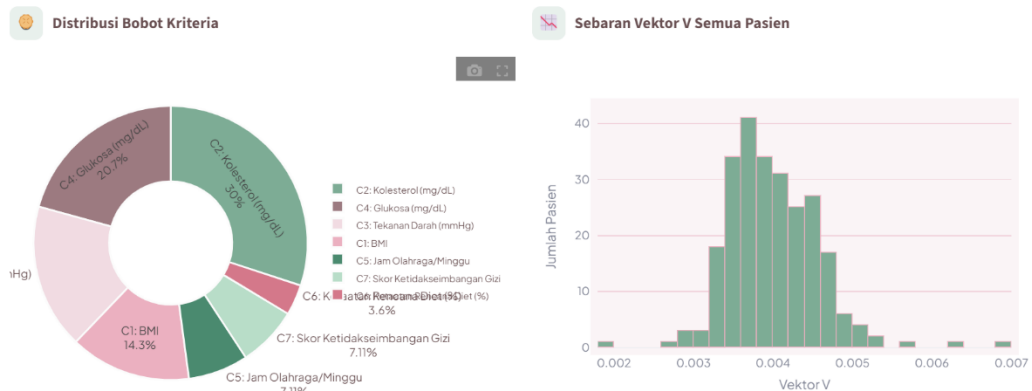
6. Menu Perankingan Pasien Berdasarkan Vektor V



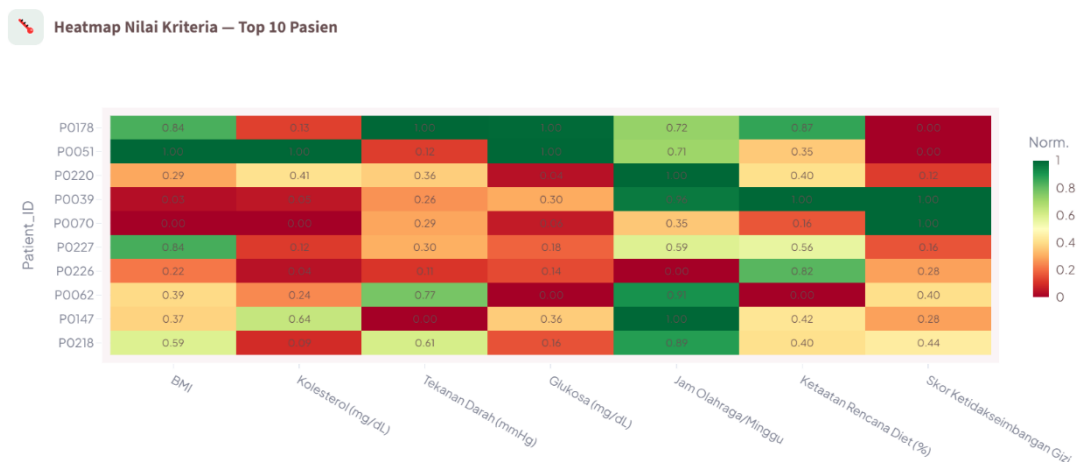
7. Menu Korelasi Bmi VS Skor Akhir (Vektor V)



8. Menu Distribusi Bobot Kriteria vs Sebar Vektor V Semua Pasien



9. Menu Heatmap Nilai Kriteria – Top 10 Pasien



BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026				
		Minggu				
		1	2	3	4	5
1	Menentukan Tema					
2	Mencari Dataset					
3	Implementasi / pembuatan program					
4	Penyusunan laporan					
5	Finalisasi proyek					

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Tazkya	123240029	- Menentukan tema - Mencari dataset - Pembuatan program - Finalisasi proyek
2	Yola	123240199	- Menentukan tema - Mencari dataset - Penguji program - Penyusunan laporan

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan dan pembahasan proyek akhir Sistem Pendukung Keputusan Penentuan Prioritas Peserta Program Diet, dapat disimpulkan bahwa sistem yang dibangun mampu membantu tenaga kesehatan atau pendamping program diet dalam menentukan peserta yang perlu diprioritaskan untuk mendapatkan intervensi diet secara lebih objektif dan terstruktur. Metode Weighted Product (WP) dapat diterapkan dengan baik dalam proses pengambilan keputusan karena mampu melakukan perhitungan dan perbandingan alternatif berdasarkan bobot kriteria yang telah ditentukan.

Sistem ini memanfaatkan beberapa kriteria penilaian, yaitu Indeks Massa Tubuh (BMI), kadar kolesterol, tekanan darah, kadar glukosa, jam olahraga per minggu, tingkat ketaatan terhadap rencana diet, serta skor ketidakseimbangan gizi. Berdasarkan kriteria tersebut, sistem menghasilkan ranking peserta yang menunjukkan tingkat prioritas dalam mengikuti program diet.

Dengan adanya proses perhitungan yang sistematis, pengguna dapat memperoleh hasil perbandingan secara lebih cepat, konsisten, dan terukur. Selain itu, sistem pendukung keputusan ini dapat membantu tenaga kesehatan dalam melakukan proses penyaringan (screening) awal terhadap banyak peserta sehingga sumber daya dan waktu pendampingan dapat dialokasikan secara lebih efektif kepada peserta yang membutuhkan perhatian lebih besar.

Oleh karena itu, proyek ini diharapkan dapat menjadi alat bantu pengambilan keputusan yang bermanfaat dalam mendukung pelaksanaan program diet yang lebih tepat sasaran dan berbasis data.

4.3 Saran

Adapun saran untuk pengembangan Sistem Pendukung Keputusan Penentuan Prioritas Peserta Program Diet di masa mendatang adalah sebagai berikut:

1. Menambahkan lebih banyak kriteria penilaian, seperti riwayat penyakit, pola tidur, tingkat stres, riwayat keluarga, dan kondisi kesehatan lainnya agar hasil perbandingan menjadi lebih akurat.
2. Menggunakan sumber data yang lebih beragam dan berasal dari data kesehatan nyata sehingga hasil yang diperoleh dapat lebih merepresentasikan kondisi peserta secara aktual.
3. Menambahkan fitur monitoring perkembangan peserta, seperti perubahan berat badan, BMI, kadar glukosa, dan tingkat kepatuhan terhadap program diet secara berkala.
4. Mengembangkan tampilan antarmuka (user interface) agar lebih menarik, interaktif, dan mudah digunakan oleh tenaga kesehatan maupun pengguna umum.

5. Mengintegrasikan sistem dengan database online atau sistem informasi kesehatan sehingga data peserta dapat tersimpan dan dikelola secara lebih efektif.
6. Mengembangkan sistem berbasis mobile (Android/iOS) atau website yang dapat diakses secara fleksibel kapan saja dan di mana saja.
7. Melakukan perbandingan dengan metode Sistem Pendukung Keputusan lainnya, seperti SAW, TOPSIS, atau MOORA, untuk mengevaluasi konsistensi dan kualitas hasil perangkingan yang dihasilkan.

DAFTAR PUSTAKA

- Kusrini. Konsep dan Aplikasi Sistem Pendukung Keputusan. Yogyakarta: Andi Offset, 2007.
- Nofriansyah, Dicky. Konsep Data Mining Vs Sistem Pendukung Keputusan. Yogyakarta: Deepublish, 2014.
- Pressman, Roger S. Rekayasa Perangkat Lunak: Pendekatan Praktisi. Yogyakarta: Andi, 2012.
- Suryadi, Kadarsah, dan M. Ali Ramdhani. Sistem Pendukung Keputusan: Suatu Wacana Struktural Idealisasi dan Implementasi Konsep Pengambilan Keputusan. Bandung: Remaja Rosdakarya, 2002.
- Turban, Efraim, Jay E. Aronson, dan Ting-Peng Liang. Decision Support Systems and Intelligent Systems. New Jersey: Pearson Education, 2005.
- Yoon, K. Paul, dan Ching-Lai Hwang. Multiple Attribute Decision Making: An Introduction. California: Sage Publications, 1995.